

捨てロボット犬を 拾った話

Java & ロボティクス 物語教科書

粗大ゴミの山で動かないロボット犬を拾った。
直したい。動かしたい。もっと賢くしたい。
これは——プログラムでロボットを育てる物語。

目次

- 第1話 拾った
- 第2話 電源を入れた
- 第3話 名前をつけた
- 第4話 蓋を開けた
- 第5話 部品を交換した
- 第6話 新しい友達
- 第7話 友達を増やした
- 第8話 頭脳を作り替えた
- 第9話 考えられるようになった
- 第10話 仲間と会話する
- 第11話 チームで動く

第1話 拾った

あらすじ

夏休みの最終日。主人公のタクミ（中学2年）は、粗大ゴミの集積所で一匹のロボット犬を見つける。

ボロボロだった。片耳は取れかけ、塗装は剥げ、動かない。

でも——目があった気がした。

タクミはそのロボット犬を家に持ち帰ることを決意する。

本文

粗大ゴミの山のとっぺんに、それは横向きに倒れていた。

「なんだこれ…」

タクミは自転車を停めて、近づいた。

犬だった。いや、ロボットの犬だ。全身が銀色のプラスチック製で、ところどころに「AIBO」の文字が見える。片方の耳は根元から折れそうになっていて、塗装はあちこち剥げている。

——動かない。

タクミはそっと触った。冷たい金属の感触。目を閉じたままのロボット犬は、まるで眠っているように見えた。

「…捨てるなら、もらっていいよ」

タクミはロボット犬を抱え上げた。思ったより軽い。自転車の荷台に無理やり縛りつけて、家まで漕いだ。

部屋に運び込んで、タクミはロボット犬を机の上に置いた。

よく見ると、首の後ろにシリアル番号っぽい刻印がある。その下に、小さな文字でこう書いてあった。

ION-CORP RS-01
Java Platform Ready

「Java…？」

学校の情報の時間に聞いたことがある。プログラムを書く言語のひとつだ。

タクミはノートパソコンを開いて、検索した。

「ロボット犬 RS-01 修理」

出てきたのは、たった一つのページ。製造元のサイトのアーカイブだった。

そこにはこう書いてあった。

RS-01 サービスマニュアル（抜粋）

本機は Java で制御されています。電源投入後、シリアル経由で JVM が起動し、`/internal/system/Main.class` が実行されます。

初期起動に必要なもの

- Java 21
- シリアル通信環境
- 本体に電源を供給できる USB-C ケーブル

「Java……授業でちょっとやっただけだけど」

タクミはリュックから USB-C ケーブルを取り出して、ロボット犬の首の横にあるポートに差し込んだ。

ノートPCが「デバイスを認識しました」と表示した。

——そして、ロボット犬の耳がピクリと動いた。

「うわっ！」

タクミは思わずのけぞった。

目が開いた。青いLEDの光が、タクミを映している。

ロボット犬はゆっくりと首を動かして、部屋の中を見回した。そして——。

耳を倒した。しょんぼりした表情に、見えた。

「バッテリー切れっぽいな…」

画面には、シリアルモニタからのメッセージが表示されていた。

```
Booting RS-01...  
JVM loaded.  
Main class not found: /internal/system/Main.class  
System halted.
```

「プログラムが……ない？」

この話で出てきたこと

- ロボット犬「RS-01」を拾った
- 動かない理由は「プログラムがないから」
- Java でプログラムを書けば動くかもしれない
- 「Main.class が見つからない」というエラーメッセージ

次回：「電源は入った。でも、プログラムがなかった。」

第2話 電源を入れた

あらすじ

ロボット犬にはプログラムが必要だ。タクミは Java の基本を調べながら、最初のプログラムを書く。

「Hello, World」ならぬ「Hello, Robot」。

目指せ——起動。

本文

「Main.class がない……」

タクミは検索しまくった。

Java のプログラムは `.java` というファイルに書いて、`javac` という道具でコンパイルすると `.class` ファイルができる。

つまり、`Main.java` を書けばいいんだ。

タクミはテキストエディタを開いて、打ち込んだ。

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("起動します");  
    }  
}
```

意味はよくわからない。でも、とにかく「起動します」って表示するプログラムらしい。

コンパイルした。

```
javac Main.java
```

エラーは出なかった。カレントディレクトリに `Main.class` ができている。

「これをロボット犬の中に送り込む…？」

サービスマニュアルをもう一度読むと、こう書いてあった。

```
ファイル転送:  
シリアル接続後、 internal/ ディレクトリに Main.class を配置してください。
```

タクミは言われた通りに、 `Main.class` をロボット犬に転送した。

```
transferred: Main.class → /internal/system/Main.class
```

「…よし、起動してみよう」

USB-C ケーブルを抜いて、もう一度差し込んだ。

ロボット犬の耳がピクリ。

目が開く。

シリアルモニタに文字が流れた。

```
Booting RS-01...  
JVM loaded.  
Main class found.  
Executing..  
  
--- 起動します ---  
  
System: 起動完了。音声モジュール未初期化。
```

ロボット犬が立ち上がった。

そして——

「……」

こっちを見ている。

「あ、あの……こんにちは？」

ロボット犬は首をかしげた。

「そうか。喋るプログラムもないんだな……」

タクミはもう一度エディタを開いた。

「喋らせたい。…どう書くんだ？」

検索すると、メソッドというものを使うらしい。「メソッド」は「動き」のこと。ロボット犬に「動き」を教えてやればいい。

```
public class Main {  
    public static void main(String[] args) {  
        greet();  
    }  
  
    static void greet() {  
        System.out.println("こんにちは");  
        System.out.println("ぼくはRS-01です");  
        System.out.println("よろしく");  
    }  
}
```

コンパイルして転送して再起動。

```
---  
こんにちは  
ぼくはRS-01です  
よろしく  
---
```

「喋った……！」

ロボット犬が「こんにちは」と言った——少なくとも、シリアルモニタに表示された。

ロボット犬は青い目をぱちぱちさせて、また首をかしげた。

この話で出てきた概念

クラス

```
public class Main {
```

プログラムを入れる「箱」。箱の名前は `Main`。

main メソッド


```
public static void main(String[] args) {
```

プログラムの「入口」。ここから実行が始まる。

メソッド呼び出し

```
greet();
```

他の「動き」を呼び出すこと。

println

```
System.out.println("こんにちは");
```

文字を表示する命令。

次回予告

ロボット犬に名前をつけたい。でも、プログラムを直すたびに `Main.java` を書き換えるのはめんどうだ。

「名前を変えるには、毎回コンパイルし直さないといけないの？」

新しいアイデアが必要だ。

第3話 名前をつけた

あらすじ

毎回プログラムを書き換えて名前を変えるのに疲れたタクミは、「名前を外から変えられる仕組み」を調べる。

そこで出会ったのが——**変数**と**クラス**の本当の意味。

本文

「毎回 `Main.java` を書き換えるの、めんどくさい…」

タクミは3回目の名前変更でそう思った。

最初は「RS-01」。次に「シロ」。そして「クロ」にしようとして——プログラムを直すのが面倒になった。

タクミは検索した。

「プログラム 値を変える 直さずに」

出てきたのは「変数（へんすう）」という言葉。

```
String name = "シロ";
```

「String name」……文字列を入れる箱？「= "シロ"」で中身を入れている？これなら、この一行だけ書き換えれば名前が変わる。

```
public class Main {
    public static void main(String[] args) {
        String name = "シロ";
        greet(name);
    }

    static void greet(String dogName) {
        System.out.println("こんにちは");
        System.out.println("ぼくは" + dogName + "です");
    }
}
```

コンパイルして転送。

```
---  
こんにちは  
ぼくはシロです  
---
```

「できた……！これで名前を変えるときは `String name = "シロ";` のとこだけ直せばいい」

でも。タクミは気づいた。

「これ、Main っていうクラスが『プログラム全体』になってて、ロボット犬自体がどこにもいない……。

ロボット犬を——『モノ』としてプログラムで表現できないかな？」

検索してたどり着いたのは、「クラスとオブジェクト」という考え方。

クラスは「設計図」。オブジェクトは「設計図から作った実物」。

```
// 設計図  
class RobotDog {  
    String name;  
}
```

```
// 実物を作る  
RobotDog myDog = new RobotDog();  
myDog.name = "シロ";
```

タクミはひらめいた。

「ロボット犬を『クラス』として設計すれば、いくらでも実物を作ったり、改造したりできるんだ！」

タクミはコードを全部書き直した。

```
class RobotDog {
    String name;

    void greet() {
        System.out.println("こんにちは");
        System.out.println("ぼくは" + name + "です");
    }
}

public class Main {
    public static void main(String[] args) {
        RobotDog myDog = new RobotDog();
        myDog.name = "シロ";
        myDog.greet();
    }
}
```

コンパイル。転送。起動。

```
---
こんにちは
ぼくはシロです
---
```

タクミは画面を見て、にやけた。

「これで……シロが『プログラムの中』に生きてるみたいになった」

シロ（ロボット犬本体）は、首をかしげたままタクミを見ている。

「お前、今プログラムの中で『オブジェクト』になってるんだぞ」

シロは——わかってないような顔をした。

知ったこと

- **変数**：値を入れる箱。 `String name = "シロ";`
- **クラス**：設計図。 `class RobotDog { ... }`
- **オブジェクト（インスタンス）**：設計図から作った実物
- **new**：実物を作る命令。 `new RobotDog()`
- **プロパティ**：オブジェクトが持つデータ。 `name`
- **メソッド**：オブジェクトが持つ動き。 `greet()`

次回予告

動くようになったシロだけど、触ってみるとガタガタだ。「中身、めっちゃ壊れてる……！」

タクミはシロのハードウェアを分解し始める。そこで見たのは——「隠された設計図」と「触ってはいけない場所」。

第4話 蓋を開けた

あらすじ

シロの調子が悪い。片耳は取れかけ、バッテリーはすぐ切れる。タクミはシロを修理しようと分解する。

そこで見たのは——保護カバーと、触ってはいけない基板。

「外から触っていい場所と、触っちゃダメな場所があるんだ…」

本文

「シロ、お手」

シロはゆっくりと前脚を上げた。——が、途中でガクンと下げた。片耳も取れかけのままぶら下がっている。

「こりゃあ、ハードウェアから直さないと」

タクミはドライバーセットを取り出した。シロを机の上に寝かせて、背中中のネジを外し始める。

カバーを開けると、中にはぎっしりと基板と配線が詰まっていた。

「すげえ……本格的だ……」

でも、どうやって直せばいいんだろう？タクミはサービスマニュアルを開いた。

カプセル化（ハードウェア版）

サービスマニュアルにはこう書いてあった。

【警告】

- 基盤A-3（バッテリー制御モジュール）は保護カバーで覆われています
- このカバーを外すと保証対象外になります
- 基盤A-3に直接アクセスする必要は通常ありません
- 充電は外側のUSB-Cポートから行ってください

つまり——

- 充電するだけなら、外側のポートにケーブルを差せばOK
- 基板A-3の中身を知らなくても、ロボット犬は使える
- 保護カバーは「触ってはいけない場所」の印

「これ、プログラムも同じじゃないか？」

タクミは昨日書いたコードを見直した。

```
class RobotDog {  
    String name;  
    int battery;  
}
```

「今、`battery` ってプログラムのどこからでも触れるようになってる。でも、バッテリーの管理って、外から適当に触られたら困るはず…」

Java には「アクセス修飾子」というものがある。`private` をつけると、そのクラスの外からは触れなくなる。

```
class RobotDog {  
    private String name;  
    private int battery;  
  
    void charge() {  
        battery = 100;  
        System.out.println("充電したよ！");  
    }  
  
    int getBattery() {  
        return battery;  
    }  
}
```

「これで、`battery` は外から直接触れなくなった。バッテリーを操作したかったら `charge()` っていうメソッドを通すしかない」

それが——**カプセル化**。

タクミは物理的にもシロを修理した。取れかけの耳を接着剤で固定し、バッテリーの接点を清掃した。

カバーを閉める。

「さて、動くか？」

電源投入。

シロが立ち上がった。耳も落ちなくなっている。

「ワン！」

シロの口から——初めて音声が出た。

「えっ！？」

「ワン！」

「しゃべった！？ いや、吠えた！？ 音声モジュール、直ったの！？」

シロはしっぽを振った——いや、しっぽはなかった。代わりに、背中のアンテナがぶるぶる震えている。

タクミは笑った。シロは首をかしげた。

知ったこと

- **private**：自分のクラスの中からは触れない
- **カプセル化**：データを隠して、必要な操作だけ公開すること
 - ハードウェアでは「保護カバー」
 - ソフトウェアでは「private」
- **getter**：値を取得するためのメソッド（`getBattery()`）
- **メリット**：中身を壊されない、後で中身を変えても外に影響しない

次回予告

シロが吠えられるようになった！ でも、吠え声はいつも同じ。「ワン」しか言わない。

「他の鳴き方もさせたいな…」 「でも、いちいちプログラムを書き換えるのもな…」

そこでタクミは「ベースの機能を引き継いで、新しい機能を追加する」方法を調べる。

第5話 部品を交換した

あらすじ

シロの吠え声は「ワン」だけ。もっと色々なバリエーションが欲しい。

タクミは「スーパーロボット犬」へのアップグレードを決意する。ベースはそのままに、新しい機能だけを追加したい——**継承**の登場。

本文

「ワン！」

シロが吠えた。元気いっぱいだ。

でも、それだけだ。いつも「ワン」。

「もう少しバリエーションが欲しいな…」 「日に何度も同じ『ワン』だと、さすがに…」

タクミは考える。

「シロの基本機能はそのままに、『声のバリエーション』だけ追加したい」 「でも、シロのクラスを直接書き換えるのは——なんか違う気がする」

サービスマニュアルの、さらに奥のページに、こんな記載があった。

RS-01 は拡張モジュールに対応しています。
専用ドーターボードを接続することで、
音声出力のバリエーションを増やすことができます。

「ドーターボード……拡張基板か！」

タクミはネットで中古の拡張基板を探した。幸い、互換品が安く出ていた。

基板をシロの背中にある拡張スロットに差し込む。カチッ、と音がした。

「これでハードウェアはOK。次はソフトウェアだ…」

継承

Java には **継承** という仕組みがある。既存のクラスをベースに、新しいクラスを作る。

```
// ベースのロボット犬
class RobotDog {
    private String name;
    private int battery;

    RobotDog(String name) {
        this.name = name;
        this.battery = 100;
    }

    void bark() {
        System.out.println(name + " : ワン!");
    }
}

// 拡張版：継承で作る
class SuperRobotDog extends RobotDog {

    SuperRobotDog(String name) {
        super(name); // ベースの初期化を呼ぶ
    }

    // 新しい機能を追加
    void barkLoudly() {
        System.out.println("ガウオーン!!");
    }

    void barkSoftly() {
        System.out.println(".....う");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        SuperRobotDog shiro = new SuperRobotDog("シロ");
        shiro.bark();           // ベースの機能（ワン！）
        shiro.barkLoudly();     // 新しい機能（ガウオーン！！）
        shiro.barkSoftly();     // 新しい機能（.....う）
    }
}
```

「ベースの機能はそのままに、新しい機能が追加できた！」

タクミはコンパイルして転送した。

「シロ、試しに」

「ワン！」

「うん。次、大声で」

「ガウオーン！！」

タクミは隣の部屋を気にした。

「……今のはデカすぎた。小さい声で」

「……う」

「完璧だ……！」

オーバーライド（上書き）

「でも待てよ。bark() 自体を変えたいときはどうするんだ？」

Java では、親クラスのメソッドを **オーバーライド（上書き）** できる。

```
class SuperRobotDog extends RobotDog {  
  
    @Override  
    void bark() {  
        System.out.println(getName() + "：ワンワンワン！（元気いっぱい）");  
    }  
}
```

`@Override` って書くと、「親のメソッドを上書きしてるよ」という意味になる。

タクミは机に突っ伏したシロの頭を撫でた。

「すごいぞシロ。お前、『継承』と『拡張』ができるようになったんだ」

シロは——撫でられていることには気づいてないようだった。バッテリー残量が少なかったからだ。

知ったこと

- **継承 (extends)**：既存のクラスをベースに新クラスを作る
- **super**：親クラスを指す。コンストラクタは `super()` で呼ぶ
- **オーバーライド**：親のメソッドを子クラスで上書きする
- **@Override**：コンパイラに「上書きしてるよ」と伝える印

次回予告

スーパーロボット犬になったシロ。でも、タクミの友達が「うちのペットロボットも動かしたい」と言い出した。別のメーカーの、別の設計のロボットだ。

「全部『ロボット犬』として同じ操作で動かせないかな…」

第6話 新しい友達

あらすじ

友達のアユムが「俺のロボット犬も動かしてくれ」と持ち込んだ。でも、それはシロとはまったく別のメーカーの、別の設計のロボットだった。

「シロと同じ『犬』として扱いたいのに、クラスが違う…」そこで登場するのが——**インターフェース**。

本文

「タクミ——！」

夏休みも終わりに近づいたある日、友達のアユムがタクミの家に飛び込んできた。

腕に抱えているのは、——

「それ、ロボット犬？」

「そう！うちの物置にあったんだけど、動かなくて！」

アユムが机に置いたのは、シロよりひと回り小さいロボット犬だった。赤いボディ。丸い目。シロとは似ても似つかないデザイン。

「何ていう機種？」

「わかんない。裏になんか書いてある」

裏返すと、刻印があった。

PET-101
by WAN-WAN Robotics

「WAN-WAN Robotics……聞いたことないメーカーだ」「シロはION-CORP製で、こっちはWAN-WAN製…」
「設計がぜんぜん違う」

タクミは PET-101 を PC に接続した。

シリアルモニタに表示された：

```
PET-101 bootloader v2.3  
Waiting for program...
```

「プログラムなし……シロと同じだ」

でも、問題があった。

シロのクラス設計と、PET-101 のクラス設計。別の設計だから、同じメソッド名かどうかもわからない。

「例えばシロは `bark()` だけど、PET-101 は `sound()` かもしれない」「操作方法がバラバラだと、プログラムを書くのが大変だ…」

インターフェース

インターフェース は、「こんなメソッドを持っていること」を約束するもの。

```
interface RobotPet {  
    void bark();  
    void walk();  
    void greet();  
}
```

このインターフェースを「約束」として、シロにもPET-101にも守らせる。

```
class ION_RS01 implements RobotPet {
    public void bark() { System.out.println("ワン!"); }
    public void walk() { System.out.println("歩く"); }
    public void greet() { System.out.println("こんにちは!"); }
}

class WAN_PET101 implements RobotPet {
    private boolean happy = true;

    public void bark() {
        if (happy) {
            System.out.println("キャンキャン!");
        } else {
            System.out.println(".....");
        }
    }

    public void walk() { System.out.println("ちょこちょこ歩く"); }
    public void greet() { System.out.println("ピー！（挨拶音）"); }
}
```

そうすると——こんなことができる。

```
public class Main {
    public static void main(String[] args) {
        RobotPet[] pets = {
            new ION_RS01(),
            new WAN_PET101()
        };

        for (RobotPet p : pets) {
            p.greet();
            p.bark();
            p.walk();
            System.out.println("---");
        }
    }
}
```

「両方とも `RobotPet` っていう同じ型として扱えるんだ！」

コンパイルして転送。

「動け！」

シリアルモニタに結果が流れた。

こんにちは！
ワン！
歩く

ピー！（挨拶音）
キャンキャン！
ちょちょこ歩く

」

「動いた...！ 2匹とも！」

アユムが拍手した。

シロと赤いロボット犬が、机の上で向かい合っている。
シロのほうが大きくて、赤いほうが小さい。
お互いをじっと見つめている。

「.....喧嘩しないといいけど」

「大丈夫だって。どっちもペットだろ」

知ったこと

- **インターフェース (interface) **：メソッドの「約束事」だけを定義する
- **implements**：「このインターフェースを守ります」という宣言
- インターフェースを使うと、違うクラスも同じ型として扱える
- 継承は「is-a」（～の一種）、インターフェースは「～できる」（能力）

次回予告

シロと赤いロボット犬（アユムが「ベニ」と名付けた）は仲良さそうだ。

でも、タクミは気づく。

「シロに『吠えろ』って命令したときの動きと、ベニに『吠えろ』って命令したときの動き—結果が違う」

同じ命令なのに、動きが違う。

これって—すごいことじゃないか？

第7話 友達を増やした

あらすじ

シロとベニは「同じ命令なのに、違う動き」をする。これが、もっとたくさんのロボットになるとどうなる？

3匹目、4匹目……どんどん増えていくロボット犬たち。タクミは気づく。

「呼び出す側は『何のロボットか』をぜんぜん気にしてない！」

本文

ベニが来てから、タクミの部屋は賑やかになった。

シロ（ION-CORP RS-01） ベニ（WAN-WAN PET-101）

そして3匹目。今度はタクミ自身がネットオークションで見つけた。

「ジャンク品・動作未確認・現状渡し」

送料込みで500円。

届いた箱を開けると、中から現れたのは——

「……でっかい」

シロの1.5倍はある。脚は4本とも独立駆動っぽい。カメラアイが2つ。背面に刻印。

HEAVY-DUTY RT-5000
by TATSUMI HEAVY INDUSTRY

「タツミ重工……軍用？ それとも工業用？」

シリアル接続すると、独自プロトコルだった。しかし、よく見ると——同じ RobotPet インターフェースで動かせるように設計されていた。タツミ重工も、この業界標準に従っていたのだ。

```
class TATSUMI_RT5000 implements RobotPet {

    public void bark() {
        System.out.println("ヴオン！ （低周波警報音）");
    }

    public void walk() {
        System.out.println("ドシン... ドシン... （重量歩行）");
    }

    public void greet() {
        System.out.println("【起動完了】 RT-5000 運用を開始します");
    }
}
```

タクミはコードを書いた——この大きなロボットを動かすために。

「3匹になった。せっかくだから、全部同時に動かしてみよう」

```
public class Main {
    public static void main(String[] args) {
        RobotPet[] pets = {
            new ION_RS01(),      // シロ
            new WAN_PET101(),    // ベニ
            new TATSUMI_RT5000() // ???（まだ名前がない）
        };

        String[] names = {"シロ", "ベニ", "ゴウ"};

        for (int i = 0; i < pets.length; i++) {
            System.out.println("--- " + names[i] + " ---");
            pets[i].greet();
            pets[i].bark();
            pets[i].walk();
        }
    }
}
```

コンパイル。転送。実行。

```
--- シロ ---  
こんにちは！  
ワン！  
歩く  
--- ベニ ---  
ピー！（挨拶音）  
キャンキャン！  
ちょちょこ歩く  
--- ゴウ ---  
【起動完了】 RT-5000 運用を開始します  
ヴォン！（低周波警報音）  
ドシン... ドシン...（重量歩行）
```

タクミは声を出して笑った。

「すげえ……3匹とも違う動きしてる……！」

呼び出しているプログラムは同じなのに。

`pets[i].bark()` の一行で、シロは「ワン」、ベニは「キャンキャン」、ゴウは「ヴォン」。

これが——多態性（ポリモーフィズム）。

ゴウ（RT-5000）が動き出した。重い足音を立てて、部屋の中を2歩、3歩。シロとベニは、その巨体を見上げている。

「……これは、もっと面白くなりそうだ」

知ったこと

- **多態性（ポリモーフィズム）**：同じメソッド呼び出しでも、オブジェクトによって動きが違う
- **アップキャスト**：`RobotPet p = new ION_RS01();` のように、具体的な型を抽象型に入れること
- **動的ディスパッチ**：実行時に「どのメソッドを呼ぶか」が決まる仕組み
- **多態性の3条件**：
 1. 継承orインターフェース実装がある
 2. メソッドがオーバーライドされている
 3. 親クラス/インターフェース型で参照する

- 多態性がなかった世界：

```
// 多態性がないと、ロボットの種類ごとにif文が必要
if (p instanceof ION_RS01) ((ION_RS01)p).bark();
else if (p instanceof WAN_PET101) ((WAN_PET101)p).bark();
// 新しい機種が増えるたびにこのif文を書き換える...
```

次回予告

シロとベニとゴウ。3匹のロボット犬がタクミの部屋に揃った。

でも——どれも「同じ動き」しかできない。賢くない。自分で考えない。プログラムに書かれた通りにしか動かない。

「もっと『自分で考えて動く』ロボットにできないかな…」

タクミは「頭脳」を作り替えることを決意する。

第8話 頭脳を作り替えた

あらすじ

「プログラム通りにしか動かない」ロボットたち。タクミは「自分で判断する頭脳」を作ろうとする。

でも——性格はひとつじゃない。好奇心旺盛な子、慎重な子、やんちゃな子。

「頭脳だけ、取り替えられたら…？」——**戦略パターン**の始まり。

本文

「シロ、お手」

シロは前脚を上げた。「ベニ、お手」ベニも前脚を上げた。「ゴウ、お手」

ゴウの重い前脚が、机の上にドンと置かれた。

「みんな『お手』できる。でもさ——」

タクミは考えた。

「自分で『何をするか』を決められたら、もっとロボットらしくない？ もっと——『生きてる』みたいに」

Brain（頭脳）の設計

タクミは新しい設計図を描き始めた。

「ロボット犬は『身体』と『頭脳』に分けよう」「身体は固定。頭脳は差し替え可能に——」

```
interface Brain {  
    String decide(String situation);  
}
```

「『状況を伝えるから、何をするか決めて』——それが頭脳」

3種類の頭脳を作った。

```
class CuriousBrain implements Brain {
    public String decide(String s) {
        return "探索する！";
    }
}

class CautiousBrain implements Brain {
    public String decide(String s) {
        return "周囲を確認する";
    }
}

class RandomBrain implements Brain {
    public String decide(String s) {
        double r = Math.random();
        return r < 0.33 ? "右に行く" :
            r < 0.66 ? "左に行く" : "その場で待つ";
    }
}
```

次に、頭脳を受け取る「身体」を作る。

```
class RobotDogBody {
    private String name;
    private Brain brain;

    RobotDogBody(String name, Brain brain) {
        this.name = name;
        this.brain = brain;
    }

    void setBrain(Brain brain) {
        this.brain = brain;
        System.out.println(name + "：頭脳を交換しました");
    }

    void act(String situation) {
        String action = brain.decide(situation);
        System.out.println(name + " は " + situation + " で " + action);
    }
}
```

「動け！」

```
public class Main {  
    public static void main(String[] args) {  
        RobotDogBody shiro = new RobotDogBody("シロ", new CuriousBrain());  
  
        shiro.act("見知らぬ部屋にいる");  
        shiro.act("物音がした");  
  
        System.out.println("--- 頭脳交換 ---");  
        shiro.setBrain(new CautiousBrain());  
  
        shiro.act("見知らぬ部屋にいる");  
        shiro.act("物音がした");  
    }  
}
```

実行結果：

```
シロ は 見知らぬ部屋にいる で 探索する！  
シロ は 物音がした で 探索する！  
--- 頭脳交換 ---  
シロ は 見知らぬ部屋にいる で 周囲を確認する  
シロ は 物音がした で 周囲を確認する
```

「頭脳を変えただけで——行動が変わった……」

タクミはしばらく、そのログを眺めた。シロを抱き上げて、目をのぞき込む。

「お前、今どっちの頭脳だ？」

シロは——やっぱりわかってない顔をした。

依存性注入

この「外部から頭脳を受け取る」設計を **依存性注入（DI）** という。

```
// 悪い例：自分で頭脳を作る（交換できない）
class RobotDogBody {
    private Brain brain = new CuriousBrain(); // 固定！
}

// 良い例：外から頭脳を受け取る（交換できる）
class RobotDogBody {
    private Brain brain;
    RobotDogBody(Brain brain) {
        this.brain = brain; // 外から注入
    }
}
```

継承よりコンポジション

「頭脳を差し替える」設計は、「**継承よりコンポジション**」の原則を体現している。

- 継承：親子関係が固定 → 後から変えられない
- コンポジション：部品を差し替え → 柔軟

知ったこと

- **戦略パターン**：アルゴリズム（頭脳）をクラス化して差し替え可能にする
- **依存性注入（DI）**：外部から部品を受け取る設計
- **コンポジション**：部品を組み合わせで機能を作る
- 「**継承よりコンポジション**」：固定より柔軟

次回予告

頭脳を作り替えたシロは「自分で判断」できるようになった。でも、判断の基準は「今の状況だけ」。

「未来を予測できたら——もっと賢くなるんじゃないか？」

タクミは次のステージへ進む。 **世界モデル**の構築。

第9話 考えられるようになった

あらすじ

「今の状況」だけでなく「この行動をしたら、次どうなるか」を予測できたら——本当に賢いロボットになる。

タクミは「世界モデル」を作り始める。

本文

頭脳を交換できるようになったシロ。でも、どこか物足りない。

「シロは『今』のことしか考えてないんだよな…」

シロがバッテリー残量25%で「探索！」を選ぶ。探索中にバッテリーが切れて、動けなくなる。そんなことが何度かあった。

「未来を予測できればいいのに。『探索したらバッテリーが10になって、動けなくなる』ってあらかじめわかってたら——」

タクミはノートに書き始めた。

State（状態）とAction（行動）

「まず、シロの『状態』をプログラムで表そう」「そして『行動』もプログラムで表す」「行動すると状態が変わる——これをプログラムにしたい」

```
class State {
    int battery;
    boolean hasBall;

    State(int battery, boolean hasBall) {
        this.battery = battery;
        this.hasBall = hasBall;
    }

    public String toString() {
        return "{battery=" + battery + ", hasBall=" + hasBall + "}";
    }
}

abstract class Action {
    abstract State execute(State current);
}

class WalkAction extends Action {
    State execute(State current) {
        return new State(current.battery - 10, current.hasBall);
    }
}

class ChargeAction extends Action {
    State execute(State current) {
        return new State(100, current.hasBall);
    }
}
```

「行動を『仮に実行』すると、未来の状態がわかる」

世界モデル

「予測する仕組み」を **世界モデル (World Model)** という。

```

class WorldModel {

    State predict(State current, Action action) {
        return action.execute(current);
    }

    int evaluate(State state) {
        int score = 0;
        score += state.battery * 2;           // バッテリーは多いほどいい
        score += state.hasBall ? 30 : 0;      // ボールがあれば嬉しい
        if (state.battery < 15) score -= 50;  // バッテリー少なすぎはダメ
        return score;
    }
}

```

「`predict` で未来を予測、`evaluate` で未来を評価。そして『一番評価の高い行動』を選べば——最適な行動になる！」

```

class PredictiveBrain implements Brain {

    WorldModel model = new WorldModel();
    Action[] actions = {new WalkAction(), new ChargeAction()};

    public String decide(String situation) {
        State current = new State(getBattery(), false);

        Action best = null;
        int bestScore = -999;

        for (Action a : actions) {
            State future = model.predict(current, a);
            int score = model.evaluate(future);
            System.out.println("予測: " + a.getClass().getSimpleName()
                               + " -> " + future + " 評価=" + score);
            if (score > bestScore) {
                bestScore = score;
                best = a;
            }
        }
        return best.getClass().getSimpleName() + " を選択";
    }
}

```

「よし、動かしてみよう！」

シロのバッテリーは25%。

```
予測: WalkAction -> {battery=15, hasBall=false} 評価=-20
予測: ChargeAction -> {battery=100, hasBall=false} 評価=200
選択: ChargeAction
```

「充電を選んだ……！」 「予測してる！未来を！」

シロは充電ドックに向かった。——いや、ロボット犬本体は相変わらずただの筐体で、動いているのはプログラム上の「判断」だけなんだけど。

でも、タクミにはシロが「考えている」ように見えた。

知ったこと

- **世界モデル (World Model)**：「行動すると状態がどう変わるか」を予測する
- **predict**：未来の状態を予測する
- **evaluate**：未来の状態を評価（点数をつける）する
- **予測→評価→選択**：3ステップで最適な行動を選べる
- 世界モデルは「多態性の現実世界版」——同じコマンドでも状況によって最適な動きが変わる

次回予告

シロは「考える」ことができるようになった。でも——1匹だとできることに限りがある。

もし、シロとベニとゴウが「会話」できたら？ 情報を共有できたら？ 「お互いの存在を知らなくても、メッセージを介して協力できる」

——そんな仕組みを作りたい。

第10話 仲間と会話する

あらすじ

シロ、ベニ、ゴウ。3匹のロボット犬は、それぞれ別々に動いている。「もし——会話できたら？」

タクミは「ROS (Robot Operating System)」の考え方を調べ始める。**ノードとトピック**。publish と subscribe。

「お互いの存在を知らなくても、情報を共有できる——」

本文

「シロが『ボール見つけた』って思っても、ベニとゴウには伝わらないんだよな」

タクミは3匹のロボット犬を見ながらつぶやいた。

シロは探索中。ベニは充電中。ゴウは待機中。それぞれが別々のことをやっている。

「人間で言えば——同じ部屋にいるのに、口をきかないで家具の方向だけ見てるようなものだ」

ネットで調べていると、タクミは「ROS」という言葉に行き着いた。

「Robot Operating System……ロボット用のOS？」 「ノード……トピック……publish……subscribe……」

publish / subscribe

ROS の基本はシンプルだ。

- **ノード**：情報を発信したり受信したりする「役割」
- **トピック**：情報の通り道
- **publish**：トピックに情報を送る
- **subscribe**：トピックから情報を受け取る

「要するに——掲示板みたいなものか」「『bark』っていう掲示板に『吠えたよ！』って書く」「見たい人はその掲示板を見てる——」

タクミはコードを書き始めた。

```
import java.util.ArrayList;

interface Subscriber {
    void onMessage(String topic, String msg);
}

interface Topic {
    void publish(String msg);
    void subscribe(Subscriber sub);
}

class BroadcastTopic implements Topic {
    ArrayList<Subscriber> subs = new ArrayList<>();
    String name;

    BroadcastTopic(String name) { this.name = name; }

    public void publish(String msg) {
        System.out.println "[" + name + "] " + msg);
        for (Subscriber s : subs) {
            s.onMessage(name, msg);
        }
    }

    public void subscribe(Subscriber sub) {
        subs.add(sub);
    }
}
```

次に、「しゃべれる犬ノード」を作る。

```
class DogNode implements Subscriber {
    String name;
    Topic barkTopic;
    Topic walkTopic;

    DogNode(String name, Topic barkTopic, Topic walkTopic) {
        this.name = name;
        this.barkTopic = barkTopic;
        this.walkTopic = walkTopic;
    }

    void bark() {
        barkTopic.publish(name + " : ワンワン!");
    }

    void walk() {
        walkTopic.publish(name + " : とことこ歩く");
    }

    public void onMessage(String topic, String msg) {
        if (!msg.contains(name)) {
            System.out.println(" [" + name + " 聞こえた] " + msg);
        }
    }
}
```

「よし、試そう！」

```
public class Main {
    public static void main(String[] args) {
        Topic barkTopic = new BroadcastTopic("bark");
        Topic walkTopic = new BroadcastTopic("walk");

        DogNode shiro = new DogNode("シロ", barkTopic, walkTopic);
        DogNode beni = new DogNode("ベニ", barkTopic, walkTopic);
        DogNode gou = new DogNode("ゴウ", barkTopic, walkTopic);

        // 全員が購読する
        barkTopic.subscribe(shiro);
        barkTopic.subscribe(beni);
        barkTopic.subscribe(gou);
        walkTopic.subscribe(shiro);
        walkTopic.subscribe(beni);
        walkTopic.subscribe(gou);

        shiro.bark();
        beni.walk();
    }
}
```

実行結果：

```
[bark] シロ：ワンワン！  
[ベニ 聞こえた] シロ：ワンワン！  
[ゴウ 聞こえた] シロ：ワンワン！  
[walk] ベニ：とことこ歩く  
[シロ 聞こえた] ベニ：とことこ歩く  
[ゴウ 聞こえた] ベニ：とことこ歩く
```

「聞こえてる……！ シロが吠えたら、ベニとゴウが『聞こえた』って表示された！」

タクミは感激した。

「シロはベニのことを知らなくていい」「ベニもシロのことを知らなくていい」「ただ『bark』っていうトピックを見てるだけで、情報が届く」

これが——疎結合（そけつごう）。

タクミはさらに `DisplayNode` という、「とにかく全部表示する係」も追加した。

```
class DisplayNode implements Subscriber {  
    String name;  
    DisplayNode(String name) { this.name = name; }  
    public void onMessage(String topic, String msg) {  
        System.out.println(" [" + name + " 表示] " + topic + ": " + msg);  
    }  
}
```

シロとベニとゴウが「会話」し始めた。少なくとも——タクミのプログラムの中では。

知ったこと

- **ROS**：ロボット用の通信フレームワーク
- **ノード**：ひとつの役割を持つプログラム
- **トピック**：メッセージの通り道
- **publish**：トピックにメッセージを送る
- **subscribe**：トピックからメッセージを受け取る
- **疎結合**：お互いの存在を知らなくても通信できる設計

次回予告

3匹は会話できるようになった。でも、「会話するだけ」では意味がない。「会話して、協力して、ひとつの目的を達成する」それが本当のチーム。

次は——マルチエージェント。複数のロボットが協調して動く世界へ。

第11話 チームで動く

あらすじ

会話できるようになった3匹。でも、まだ「協力」はしていない。

「シロがボールを見つけた」「ベニがそれを知る」「ベニがそれを拾いに行く」

この連携が、プログラムで実現できたら——そこに「チーム」が生まれる。

本文

「会話はできるようになった。でも——」

タクミは3匹のロボット犬を見渡した。

「まだ『協力』はしてないんだよな」「ただ『聞こえた』って表示してるだけだ」

タクミが欲しかったのは——「シロがボールを見つけた」「ベニがそれを知って、回収に行く」「ゴウがその間、安全を確保する」

そんな連携だ。

DogMessage

まず、ロボット犬同士でやりとりする「メッセージ」の型を決める。

```
class DogMessage {
    String sender;    // だれが
    String type;      // どんな種類のメッセージ
    String content;   // 何を伝えたいか

    DogMessage(String sender, String type, String content) {
        this.sender = sender;
        this.type = type;
        this.content = content;
    }

    public String toString() {
        return "[" + sender + "] " + type + ": " + content;
    }
}
```

メッセージの種類は：

- `found_ball` : ボールを見つけた
- `going_to_pickup` : 回収に向かう
- `danger` : 危険を検知
- `help` : 助けて

3匹の役割

```
interface DogAgent {
    void act();
    void onMessage(DogMessage msg);
    String getName();
}
```

シロ（探索担当）

```
class Explorer implements DogAgent {
    String name;
    Topic comm;

    Explorer(String name, Topic comm) {
        this.name = name; this.comm = comm;
    }

    public String getName() { return name; }

    public void act() {
        System.out.println(name + " : 探索中...");
        if (Math.random() > 0.6) {
            DogMessage msg = new DogMessage(name, "found_ball",
                "ボールを発見！位置は (12, 34)");
            comm.publish(msg.toString());
        }
    }

    public void onMessage(DogMessage msg) {
        // シロは探索に集中。メッセージは受け取るだけ
    }
}
```

ベニ（運搬担当）

```
class Carrier implements DogAgent {
    String name;
    Topic comm;
    boolean going = false;

    Carrier(String name, Topic comm) {
        this.name = name; this.comm = comm;
    }

    public String getName() { return name; }

    public void act() {
        if (going) {
            System.out.println(name + " : 荷物を運搬中...");
            going = false;
        } else {
            System.out.println(name + " : 待機中...");
        }
    }

    public void onMessage(DogMessage msg) {
        if (msg.type.equals("found_ball")) {
            System.out.println(name + " : ボールを回収に向かう!");
            going = true;
            DogMessage reply = new DogMessage(name, "going_to_pickup",
                "回収に向かっています");
            comm.publish(reply.toString());
        }
    }
}
```

ゴウ（警備担当）

```
class Guard implements DogAgent {
    String name;
    Topic comm;

    Guard(String name, Topic comm) {
        this.name = name; this.comm = comm;
    }

    public String getName() { return name; }

    public void act() {
        System.out.println(name + " : 周囲を警戒中...");
        if (Math.random() > 0.7) {
            DogMessage alert = new DogMessage(name, "danger",
                "異常なし。継続監視中");
            comm.publish(alert.toString());
        }
    }

    public void onMessage(DogMessage msg) {
        if (msg.type.equals("help")) {
            System.out.println(name + " : 救援に向かう!");
        }
    }
}
```

チーム始動

タクミは3匹を動かした。

```
public class Main {
    public static void main(String[] args) {

        Topic commTopic = new BroadcastTopic("team_channel");

        DogAgent[] team = {
            new Explorer("シロ", commTopic),
            new Carrier("ベニ", commTopic),
            new Guard("ゴウ", commTopic)
        };

        for (int step = 1; step <= 5; step++) {
            System.out.println("\n=== ステップ " + step + " ===");
            for (DogAgent agent : team) {
                agent.act();
            }
        }
    }
}
```

実行が流れた。

```
=== ステップ 1 ===  
シロ：探索中...  
ベニ：待機中...  
ゴウ：周囲を警戒中...  
[team_channel] ゴウ：異常なし。継続監視中  
  
=== ステップ 2 ===  
シロ：探索中...  
[team_channel] [シロ] found_ball: ボールを発見！位置は (12, 34)  
ベニ：ボールを回収に向かう！  
[team_channel] [ベニ] going_to_pickup: 回収に向かっています  
ゴウ：周囲を警戒中...  
  
=== ステップ 3 ===  
シロ：探索中...  
ベニ：荷物を運搬中...  
ゴウ：周囲を警戒中...
```

「協力してる……！」

タクミはモニタに映るログを見ながら、声が出た。

シロが見つけた。ベニが受け取って動いた。ゴウがその間、警戒を続けている。

「3匹が——ひとつのチームになってる」

タクミは椅子の背にもたれた。

机の上にはシロ。その隣でベニが充電中。床にはゴウが鎮座している。

どれも、最初は動かなかったロボットたちだ。拾ってきて、直して、プログラムを書いて——今、ここで、3匹のロボット犬が「チーム」として協力している。

「プログラムって——すごいな」

シロの青い目が、タクミを見ている。何かを言いたそうに、口を開けて——閉じた。

「……今、何か言おうとした？」

シロは首をかしげた。

タクミは笑った。

「よし、もっとすごいプログラムを書こう」

この物語で学んだこと

すべては、粗大ゴミの山で動かないロボット犬を拾ったことから始まった。

話	タイトル	学んだこと
1	拾った	Java プログラムが必要だと知った
2	電源を入れた	クラスと main メソッド、println
3	名前をつけた	変数、クラスとオブジェクト
4	蓋を開けた	カプセル化、private
5	部品を交換した	継承、オーバーライド
6	新しい友達	インターフェース
7	友達を増やした	多態性（ポリモーフィズム）
8	頭脳を作り替えた	戦略パターン、DI
9	考えられるようになった	世界モデル
10	仲間と会話する	pub/sub、ROS
11	チームで動く	マルチエージェント

おわり

この後も、物語は続く。新しいロボット。新しい機能。新しい仲間。

プログラムは——無限に広がっている。